

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)**

РАСШИРЕННЫЙ КОНСПЕКТ ЛЕКЦИИ № 3

МАССИВЫ И ДИНАМИЧЕСКИЕ МАССИВЫ: ПРЕДСТАВЛЕНИЕ В ПАМЯТИ, ОПЕРАЦИИ И АМОРТИЗИРОВАННЫЙ АНАЛИЗ

по дисциплине «Алгоритмы и структуры данных»

Направление подготовки: 03.03.01 Прикладные математика и физика

Направленность (профиль): 03.03.01 Прикладные физика и математика

Семестр: 7 · Объем лекции: 2 академических часа

Компетенция: ПК-4 · Индикатор: ПК-4.4

Ростов-на-Дону

2026

1. Паспорт лекции

Позиция	Содержание
Место в РПД	Раздел 2. Линейные структуры данных и абстрактные типы данных; первая из двух лекций раздела.
Цель	Сформировать понимание устройства массивов и динамических массивов и научить выбирать их с учетом стоимости операций, памяти и характера физико-математических данных.
Планируемый результат	Обучающийся объясняет адресацию и локальность массива, получает оценки операций, реализует динамический массив с геометрическим расширением и обосновывает амортизированную стоимость добавления.
Связь с ПК-4.4	Использует современные прикладные и программные средства при решении задач профессиональной деятельности: выбирает представление последовательностей, применяет стандартные контейнеры и проверяет их ресурсные свойства.
Предварительная подготовка	Лекции № 1–2: спецификация, модель вычислений, корректность, O -, Ω -, Θ -оценки и экспериментальная проверка трудоемкости.
Текущий контроль	Устный разбор операций, трассировка расширения, мини-задача на амортизированную оценку и домашний вычислительный эксперимент.

2. Планируемые результаты по уровням

Уровень	После лекции обучающийся демонстрирует
Базовый	Различает логический размер и емкость; объясняет непрерывное размещение; указывает $\Theta(1)$ для доступа по индексу и $\Theta(n)$ для вставки в начало массива.
Средний	Реализует операции динамического массива, выводит их худшие и амортизированные оценки, сравнивает Python list, C++ std::vector и NumPy ndarray по назначению и модели хранения.
Продвинутый	Выбирает коэффициент роста и стратегию управления памятью под ограничения задачи; связывает локальность с иерархией памяти; проектирует воспроизводимый эксперимент без смешения алгоритмических и платформенных эффектов.

3. Рекомендуемая организация 90 минут

Этап	Время	Деятельность
Проблемная постановка	0–8 мин	Почему доступ по индексу быстр, а рост массива иногда вызывает скачок времени.
Модель массива	8–25 мин	Непрерывная память, адрес элемента, инварианты и стоимость базовых операций.
Динамический массив	25–47 мин	Размер, емкость, append, перенос элементов и геометрическое расширение.
Амортизированный анализ	47–65 мин	Агрегатный метод и метод условных единиц; отличие от среднего случая.
Инженерный выбор	65–78 мин	Локальность, накладные расходы, Python/C++/NumPy и научные данные.
Мини-кейс	78–86 мин	Буфер отсчетов: выбор структуры и протокол сравнения.
Закрепление	86–90 мин	Выходной вопрос и постановка домашнего задания.

4. Понятийный аппарат

Понятие	Рабочее определение
Массив	Последовательность однотипных элементов, размещенных в логически смежных позициях и доступных по целочисленному индексу.

Понятие	Рабочее определение
Динамический массив	Абстракция изменяемой последовательности, реализованная массивом большей емкости с периодическим перераспределением.
Логический размер n	Число хранимых элементов, доступных клиенту структуры.
Емкость C	Число элементов, для которых уже выделено хранилище; инвариант: $0 \leq n \leq C$.
Амортизированная стоимость	Гарантированная средняя стоимость одной операции по любой последовательности операций, без вероятностной модели входов.
Локальность	Свойство программы повторно обращаться к близким данным и тем самым эффективно использовать уровни памяти и кэш.

5. Массив как модель адресуемой последовательности

Пусть базовый адрес массива равен B , размер одного элемента — s байт, а нижняя граница индекса — 0 . Адрес элемента i вычисляется формулой $\text{addr}(A[i]) = B + i \cdot s$. В RAM-модели арифметика адреса и одно обращение считаются постоянным числом операций, поэтому доступ по корректному индексу имеет стоимость $\Theta(1)$.

Логическая схема непрерывного размещения

индекс:	0	1	2	3	4
адрес:	B	$B+s$	$B+2s$	$B+3s$	$B+4s$
значение:	x_0	x_1	x_2	x_3	x_4

Предусловие доступа. $0 \leq i < n$. Формула адреса объясняет скорость операции, но не проверяет допустимость индекса и не гарантирует безопасность конкретной реализации.

6. Инварианты и контракт массива

- элементы занимают позиции $0, \dots, n-1$ и сохраняют заданный порядок;
- индекс однозначно определяет позицию элемента;
- для фиксированного массива длина не меняется после создания;
- для динамического массива выполнено $0 \leq n \leq C$, а элементы клиента находятся в первых n позициях;
- операции не должны раскрывать клиенту незаполненную часть емкости и нарушать порядок без явного контракта.

Контракт операции $GET(i)$: предусловие $0 \leq i < n$; постусловие — возвращено значение, хранимое в позиции i , структура не изменена. Контракт $INSERT(i, x)$: $0 \leq i \leq n$; после операции x занимает позицию i , прежние элементы $i, \dots, n-1$ сдвинуты вправо, размер увеличен на единицу.

7. Стоимость операций фиксированного массива

Операция	Худшая стоимость	Причина
Доступ/замена $A[i]$	$\Theta(1)$	Адрес вычисляется непосредственно.
Последовательный просмотр	$\Theta(n)$	Каждый из n элементов читается хотя бы один раз.
Поиск без упорядоченности	$\Theta(n)$	В худшем случае проверяются все элементы.
Вставка/удаление в позиции i	$\Theta(n-i)$	Сдвигаются элементы хвоста; в начале — $\Theta(n)$.

Операция	Худшая стоимость	Причина
Копирование массива	$\Theta(n)$	Переносится n элементов; для объектов важна стоимость копирования/перемещения.

Не смешивать. стоимость поиска позиции и стоимость изменения структуры. Если сначала нужен линейный поиск, а затем сдвиг, полная операция остается $\Theta(n)$, но содержит два разных этапа.

8. Динамический массив: состояние и операция APPEND

Динамический массив хранит ссылку на буфер, логический размер n и емкость C . Если $n < C$, новый элемент записывается в позицию n . Если $n = C$, выделяется новый буфер емкости $C' > C$, переносятся n элементов, старый буфер освобождается, затем добавляется новый элемент.

Псевдокод геометрического расширения

```

APPEND(D, x)
  if D.n = D.C
    Cnew ← max(1, 2·D.C)
    Bnew ← allocate(Cnew)
    for i ← 0 to D.n - 1
      Bnew[i] ← D.B[i]
    D.B ← Bnew; D.C ← Cnew
  D.B[D.n] ← x
  D.n ← D.n + 1

```

Одна операция с расширением имеет худшую стоимость $\Theta(n)$, поскольку переносит накопленные элементы. Однако такие операции редки: после увеличения емкости появляется запас свободных позиций.

9. Почему увеличение емкости на единицу неэффективно

Если при каждом заполнении выделять ровно одну дополнительную позицию, последовательность из N добавлений потребует $0 + 1 + \dots + (N-1) = N(N-1)/2$ переносов. Суммарная стоимость равна $\Theta(N^2)$, а амортизированная стоимость одного APPEND — $\Theta(N)$.

При удвоении емкости копируются $1 + 2 + 4 + \dots + 2^k < 2N$ элементов до достижения размера N . Вместе с N обычными записями суммарная работа $\Theta(N)$, поэтому амортизированная стоимость APPEND равна $\Theta(1)$.

Главный вывод. редкая дорогая операция допустима, если ее стоимость распределяется по длинной последовательности дешевых операций и для любой последовательности существует суммарная гарантия.

10. Амортизированный анализ не является средним случаем

Подход	Что усредняется	Требуется распределение?	Результат
--------	-----------------	--------------------------	-----------

Подход	Что усредняется	Требуется распределение?	Результат
Средний случай	Стоимость по случайным входам одного размера.	Да	Математическое ожидание.
Амортизированный	Суммарная стоимость последовательности операций.	Нет	Гарантия на любую последовательность.

В агрегатном методе вычисляют полную стоимость m операций и делят ее на m . В методе условных единиц дешевой операции назначают завышенную плату, а остаток трактуют как запас для будущего переноса. Потенциальный метод формализует запас функцией состояния $\Phi(D)$ и задает амортизированную стоимость $\hat{a}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.

11. Коэффициент роста: время против памяти

Геометрический рост $C' = \lceil \alpha C \rceil$ при $\alpha > 1$ сохраняет амортизированную стоимость добавления $\Theta(1)$. Чем ближе α к 1, тем меньше запас памяти после расширения, но тем чаще выполняются выделения и переносы. Большой α уменьшает частоту переносов, но может оставить значительную неиспользуемую емкость.

Решение	Преимущество	Риск/ограничение
Малый α	Меньше пиковый запас памяти.	Больше перераспределений и копирований.
Большой α	Реже дорогие расширения.	Больше неиспользуемой емкости.
reserve/предвыделение	Устраняет расширения до ожидаемого размера.	Нужно обоснованно оценить объем данных.
Уменьшение емкости	Возвращает память после сокращения.	Без гистерезиса возможны частые рост/сжатие.

12. Локальность и фактическая производительность

Смежное размещение делает последовательный просмотр предсказуемым для аппаратной подкачки и кэша. Поэтому два алгоритма с одинаковой Θ -оценкой могут заметно различаться по времени. Асимптотика не исчезает: она задает рост, а модель памяти объясняет постоянные множители и переходы между режимами.

- пространственная локальность: после $A[i]$ вероятно обращение к $A[i+1]$;
- временная локальность: недавно использованный элемент читается повторно;
- упакованные числовые значения уменьшают объем памяти и число косвенных обращений;
- список ссылок на объекты может логически быть последовательностью, но физически данные размещены раздельно.

13. Одна абстракция — разные реализации

Средство	Модель	Когда уместно
Python list	Динамический массив ссылок на объекты; append и pop с конца удобны, операции в начале требуют сдвига.	Гетерогенные или объектные данные, общая алгоритмическая разработка.
C++ std::vector<T>	Непрерывное хранение T; size и capacity разделены; reserve управляет емкостью.	Типизированные данные, контроль размещения и стоимости перемещения.
NumPy ndarray	Однородный многомерный массив с dtype, shape и strides; возможны представления без копии.	Векторизованные численные вычисления и массивы измерений.

Терминологическая осторожность. Python list в учебной задаче часто играет роль динамического массива, а не связного списка. Название библиотечного типа не заменяет анализа его реализации и контракта.

14. Мини-кейс: буфер отсчетов физического эксперимента

Датчик формирует последовательность отсчетов температуры. После завершения опыта выполняются фильтрация, вычисление разностей соседних значений и построение спектра. Число отсчетов приблизительно известно по частоте дискретизации и длительности опыта.

- для накопления заранее вызвать `reserve(N)` либо создать числовой массив требуемого размера;
- для последовательной обработки использовать непрерывный буфер и проход слева направо;
- для онлайн-окна фиксированной длины не удалять первый элемент динамического массива на каждом шаге — выбрать кольцевой буфер или дек;
- зафиксировать dtype: переход от 64 к 32 битам уменьшает память, но меняет точность и диапазон;
- в эксперименте отдельно измерять выделение памяти, заполнение и собственно вычисление.

15. Экспериментальная проверка

Предложим две гипотезы: последовательный просмотр имеет $\Theta(n)$, а серия APPEND с геометрическим ростом — $\Theta(n)$ суммарно. Следует измерять геометрическую сетку размеров, выполнять прогрев и повторы, сохранять медиану и разброс. Для анализа расширений полезнее счетчик перенесенных элементов, чем только время в секундах.

Схема воспроизводимого эксперимента

```
for N in 2^10, ..., 2^24:
    D ← empty_dynamic_array()
    moved ← 0
    repeat N times:
        moved ← moved + APPEND_WITH_COUNTER(D, value)
    save N, moved, capacity(D), time
```

Интерпретация. скачки отдельных APPEND ожидаемы и не опровергают $\Theta(1)$ амортизированно. Проверяется суммарная стоимость последовательности, а не максимальная стоимость одной операции.

16. Вопросы для взаимодействия

- Массив содержит $n = 9$ элементов при $C = 16$. Сколько APPEND выполнятся без расширения? Что произойдет при восьмом?
- Почему INSERT(0, x) остается $\Theta(n)$, даже если свободная емкость уже есть?
- Суммарное число переносов при удвоениях 1, 2, 4, 8, 16 меньше какого выражения через N?
- Как изменится выбор структуры, если данные поступают бесконечным потоком, а нужны только последние k отсчетов?

- Какие измерения отделят влияние алгоритма роста от работы менеджера памяти и кэша?

17. Типичные ошибки

- считать, что любой `append` всегда имеет худшую стоимость $\Theta(1)$;
- называть амортизированную оценку ожидаемой или вероятностной;
- сравнивать структуры только по одной операции, игнорируя полный профиль рабочей нагрузки;
- принимать Python list за связный список из-за слова `list`;
- удалять первый элемент массива в цикле обработки окна и получать скрытую $\Theta(n^2)$ стоимость;
- измерять одновременно генерацию данных, перераспределение, вычисление и вывод результата.

18. Контрольные вопросы

1. Как вычисляется адрес $A[i]$ и при каких предположках доступ имеет $\Theta(1)$?
2. Чем логический размер отличается от емкости?
3. Какие инварианты поддерживает динамический массив?
4. Почему вставка в начало массива требует $\Theta(n)$?
5. Как работает `APPEND` при $n < C$ и при $n = C$?
6. Почему рост емкости на единицу приводит к $\Theta(N^2)$ суммарной работе?
7. Как геометрический рост дает $\Theta(1)$ амортизированно?
8. Чем амортизированный анализ отличается от среднего случая?
9. Какие компромиссы задает коэффициент роста α ?
10. Для чего используется `reserve`?
11. Почему немедленное сжатие после удаления может быть неэффективно?
12. Как локальность влияет на последовательный просмотр?
13. Чем Python list отличается от NumPy ndarray?
14. Когда кольцевой буфер лучше динамического массива?
15. Как экспериментально проверить суммарную стоимость серии `APPEND`?

19. Домашнее задание

1. Реализовать `DynamicArray` с полями `buffer`, `size` и `capacity` и операциями `get`, `set`, `append`, `insert` и `remove`. Для каждой операции записать контракт и худшую оценку.
2. Добавить счетчики выделений и перенесенных элементов. Выполнить N добавлений для $N = 2^8, \dots, 2^{20}$ при стратегиях $C+1$, $\lceil 1.5C \rceil$ и $2C$; построить таблицу суммарных переносов.
3. Сравнить три стратегии по времени, пиковой емкости и доле незанятой памяти. Отделить данные счетчиков от платформенного времени и сформулировать вывод.
4. Для последовательности отсчетов предложить представление в Python list, array/NumPy ndarray или `std::vector`; явно указать тип элемента, ожидаемый объем и операции рабочей нагрузки.
5. Подготовиться к лекции № 4: объяснить, почему известная позиция позволяет вставить элемент в связный список без сдвига остальных элементов, но не гарантирует быстрый поиск позиции.

20. Итоги лекции

- Прямой доступ массива следует из вычисления адреса элемента и корректного индекса.
- Вставка и удаление внутри массива требуют сдвига хвоста.
- Динамический массив разделяет логический размер и емкость и периодически переносит элементы.

- Геометрическое расширение обеспечивает $\Theta(1)$ амортизированно для APPEND, хотя отдельное расширение стоит $\Theta(n)$.
- Амортизированная гарантия не использует вероятностное распределение.
- Инженерный выбор учитывает локальность, тип элементов, память и полный профиль операций.

21. Основная и дополнительная литература

- Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 4th ed. MIT Press, 2022. Разделы об амортизированном анализе.
- Morin P. Open Data Structures. Array-Based Lists. Открытый учебник: https://opendatastructures.org/ods-python/2_Array_Based_Lists.html (дата обращения: 07.09.2026).
- Python Software Foundation. Python 3.14 Documentation. Data Structures: <https://docs.python.org/3/tutorial/datastructures.html> (дата обращения: 07.09.2026).
- cppreference.com. std::vector: <https://en.cppreference.com/w/cpp/container/vector> (дата обращения: 07.09.2026).
- NumPy Developers. numpy.ndarray: <https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html> (дата обращения: 07.09.2026).
- Актуализированная РПД дисциплины «Алгоритмы и структуры данных» для направления 03.03.01, редакция от 06.09.2026.

22. Пояснения к презентации лекции № 3

Этот раздел сопровождает 26 слайдов презентации «Массивы и динамические массивы» и раскрывает ход объяснения, который нельзя уместить на экране. Номера ниже совпадают с номерами слайдов переработанной презентации. На каждом шаге возвращайтесь к четырем вопросам: что обещает операция, какой инвариант сохраняется, сколько действий она выполняет и как это проверить на данных.

Слайд 1. Тема и постановка

Напомните вывод лекции 2: асимптотическая оценка имеет смысл только после выбора модели операций. Сегодня такой моделью служит адресуемая последовательность. В конце занятия студент должен суметь объяснить скачок времени при добавлении элемента, не называя всю серию добавлений квадратичной.

Слайд 2. Проблемный вопрос

Покажите два действия над одной последовательностью: чтение $A[i]$ и APPEND при заполненном буфере. Первое вычисляет адрес и читает ячейку; второе вынуждено выделить новый буфер и перенести уже накопленные элементы. Попросите предсказать, означает ли дорогой отдельный APPEND дорогую длинную серию. Ответ отложите до слайдов 16–18.

Слайд 3. Проверяемые результаты

К концу лекции студент выводит адрес элемента при заданных B , s и i ; записывает инвариант $0 \leq n \leq C$; оценивает сдвиг при вставке; объясняет худшую и амортизированную стоимость APPEND; выбирает библиотечный контейнер для конкретного потока измерений. Эти действия содержательно раскрывают индикатор ПК-4.4.

Слайд 4. Логика занятия

Причинная цепочка начинается с физического размещения элементов, переходит к стоимости операций и заканчивается выбором структуры для рабочей нагрузки. Уточните, что «массив» может обозначать и абстракцию индексируемой последовательности, и конкретный непрерывный блок памяти; в оценках ниже важна именно модель реализации.

Слайд 5. Фиксированный и динамический массив

Оба варианта позволяют получить элемент по индексу за постоянное число шагов. У фиксированного массива число доступных позиций определено заранее. Динамический массив поддерживает изменяемый логический размер за счет буфера с запасом. Слово «динамический» не означает, что каждый элемент хранится в отдельном узле.

Слайд 6. Состояние и инвариант

Разведите n — число видимых элементов — и C — число выделенных позиций. Для состояния $n = 5$, $C = 8$ позиции 0–4 содержат последовательность, а позиции 5–7 составляют резерв и недоступны по контракту GET. Инвариант $0 \leq n \leq C$ проверяется до и после каждой публичной операции. Задайте аудитории вопрос: допустим ли GET(5)?

Слайд 7. Адресация как модель

При нулевой нижней границе и одинаковом размере элементов s адрес i -го элемента равен $B + i \cdot s$. Для $B = 1000$, $s = 8$, $i = 3$ получается 1024. Постоянное число арифметических действий дает $\Theta(1)$ в RAM-модели; реальная задержка доступа может зависеть от кэша и размещения данных.

Слайд 8. Разбор формулы адреса

Покажите на схеме, что шаг между соседними элементами равен s байтам: адреса B , $B+s$, $B+2s$ и далее. Если элемент имеет переменную длину, простое произведение $i \cdot s$ неприменимо без дополнительного индекса. Свяжите этот вывод с дальнейшим сравнением `std::vector` и Python `list`: у последнего подряд лежат ссылки, а сами объекты могут находиться в разных областях памяти.

Слайд 9. Контракт индексного доступа

Запишите тройку Хоара: при $0 \leq i < n$ операция `GET(A, i)` возвращает прежнее $A[i]$ и не меняет массив. Проверка границы имеет постоянную стоимость, но она принципиальна для корректности. $\Theta(1)$ не разрешает обращение к несуществующему элементу и ничего не говорит о поведении при нарушенном предусловии.

Слайд 10. Цена сдвига

При вставке в известную позицию i нужно освободить ячейку, сдвинув $n-i$ элементов вправо, начиная с конца. Если идти от i к концу, значения будут перезаписаны до переноса. Удаление с сохранением порядка сдвигает $n-i-1$ элементов влево. Для начала последовательности обе операции имеют $\Theta(n)$, для конца без роста — $\Theta(1)$.

Слайд 11. Локальность памяти

При последовательном чтении соседние значения часто попадают в одну или соседние кэш-линии. Это улучшает фактическое время обхода, хотя число прочитанных элементов остается $\Theta(n)$. Сравнение реализаций должно учитывать тип элемента, косвенные обращения, пропускную способность памяти и то, помещаются ли данные в кэш.

Слайд 12. Поиск позиции и вставка

Если индекс i уже известен, стоимость определяется сдвигом. Если нужно сначала найти значение или место по условию, прибавляется цена поиска. Для неупорядоченного массива она может составлять $\Theta(n)$. Разделяйте два этапа при анализе: поиск не исчезает от наличия свободной емкости, а свободная емкость не избавляет от сдвига.

Слайд 13. Трассировка расширения

При $n = C$ алгоритм выделяет новый буфер $C' > C$, переносит элементы с индексами $0, \dots, n-1$ в том же порядке, затем записывает новый элемент в позицию n и увеличивает n . До замены рабочего буфера старые данные должны оставаться доступными; в реальной реализации нужно отдельно учитывать ошибки выделения и копирования. После успешного `APPEND` инвариант $0 \leq n \leq C$ и порядок элементов сохраняются.

Слайд 14. Два режима APPEND

Если $n < C$, достаточно записи в `buffer[n]` и увеличения размера — $\Theta(1)$. При $n = C$ переносится n элементов, поэтому худшая стоимость данного вызова $\Theta(n)$. Попросите студентов указать, какое условие проверяется первым и почему новый элемент нельзя записать в прежний буфер до расширения.

Слайд 15. Смысл амортизации

Амортизированная оценка относится к последовательности операций и не предполагает случайного распределения входов. Она допускает дорогой вызов, если суммарная стоимость

длинной серии ограничена. Для N добавлений геометрически растущий массив выполняет N обычных записей и ограниченное число переносов.

Слайд 16. Рост на единицу

При стратегии $C' = C+1$ каждое заполнение вызывает перенос всего накопленного массива. Для N добавлений число переносов равно $0+1+\dots+(N-1) = N(N-1)/2$. Значит, серия стоит $\Theta(N^2)$, а стоимость на операцию после деления на N равна $\Theta(N)$. Покажите, что этот вывод касается стратегии роста, а не самой возможности прямого доступа.

Слайд 17. Удвоение емкости

При емкостях 1, 2, 4, 8, ... копирование происходит только при заполнении. Сумма перенесенных элементов до размера N меньше $2N$, поэтому вместе с N записями серия стоит $\Theta(N)$. Деление суммарной верхней границы на N дает $\Theta(1)$ амортизированно. Нижняя граница $\Theta(N)$ очевидна: каждый из N элементов требуется хотя бы раз записать.

Слайд 18. Методы анализа

В агрегатном методе оценивают сумму фактических затрат; именно его достаточно для доказательства на слайдах 16–17. Метод условных единиц заранее взимает с дешевых операций плату на будущие переносы. Потенциальный метод описывает запас как функцию состояния структуры. Все три подхода дают гарантии без вероятностной модели и требуют согласованного определения стоимости элементарных действий.

Слайд 19. Переход к выбору

Оценка $\Theta(1)$ амортизированно не задает единственный коэффициент роста. Для инженерного решения укажите объем данных, допустимый пик памяти, цену копирования элемента и требуемую предсказуемость задержки. При жестком ограничении времени одного вызова периодическое $\Theta(n)$ расширение может быть неприемлемо, даже когда общая работа линейна.

Слайд 20. Коэффициент роста и reserve

При $C' = \lceil \alpha C \rceil$ и постоянном $\alpha > 1$ сохраняется геометрическая сумма переносов и $\Theta(1)$ амортизированно для APPEND. Небольшое α уменьшает свободный резерв, но повышает число расширений. Если объем измерений N известен заранее, предварительное резервирование емкости N устраняет перераспределения во время накопления; его польза зависит от точности прогноза и ограничений памяти.

Слайд 21. Три модели хранения

Python list — динамический массив ссылок на объекты; он удобен для общей последовательности объектов. `std::vector<T>` хранит элементы T непрерывно и позволяет управлять емкостью через `reserve`. NumPy ndarray хранит однородные численные данные и описывает их через `dtype`, `shape` и `strides`. Сравнивайте их на одной задаче, явно указав тип, операции и объем данных.

Слайд 22. Протокол эксперимента

Для размеров $N = 2^8, 2^9, \dots$ создайте пустой массив, выполните N добавлений и запишите число перенесенных элементов, итоговую емкость и время серии. Счетчик непосредственно проверяет математическую модель; время дополнительно зависит от среды. Повторите замеры, отделите прогрев от измерения, сохраните версии инструментария и сравнивайте одинаковые входы.

Слайд 23. Диагностика ошибок

Предложите исправить каждую неверную фразу. «Каждый APPEND имеет худшую стоимость $\Theta(1)$ » заменяется на « $\Theta(1)$ амортизированно при геометрическом росте». Python list не является связным списком. Для окна последних k отсчетов удаление первого элемента массива на каждом шаге дает многократные сдвиги; кольцевой буфер или дек соответствует такому профилю операций лучше.

Слайд 24. Самостоятельная трассировка

Пусть $n = C = 4$ и буфер хранит $\tau_1, \dots, \tau_4$. При APPEND(τ_5) и удвоении емкости новый C равен 8, переносятся четыре старых элемента, τ_5 записывается в позицию 4, итоговый размер равен 5. Этот вызов имеет $\Theta(n)$ в худшем случае. Его доля в произвольной длинной серии при той же стратегии входит в $\Theta(1)$ амортизированно. Спросите, сколько следующих добавлений пройдет без расширения: три.

Слайд 25. Выходной контроль

Попросите студентов ответить письменно на пять вопросов слайда без обращения к определениям. Особенно важен четвертый: амортизированная оценка усредняет стоимость по операциям выбранной серии, а средний случай — по вероятностной модели входов. Ошибки в ответах определяют, к каким фрагментам вернуться перед изучением связных списков.

Слайд 26. Итог и переход

Вернитесь к первому вопросу: прямой доступ и редкие дорогие расширения следуют из разных свойств одной структуры. Формула адреса объясняет GET; запас емкости и геометрический рост объясняют суммарную цену APPEND. Следующая лекция сравнит эту модель со связным хранением, стеком, очередью и деком, где меняются цены локального изменения и доступа по индексу.

23. Сквозной разбор для преподавателя

Сохранение порядка при расширении

Предусловие: динамический массив D содержит n элементов в позициях $0, \dots, n-1$ и $n = C$. После выделения буфера емкости $C' > C$ цикл переносит элементы по возрастанию индекса. Инвариант цикла: перед итерацией i новый буфер уже содержит точные копии старых элементов в позициях $0, \dots, i-1$; старый буфер и логический размер еще не изменены. При $i = 0$ утверждение истинно для пустого префикса. Один перенос сохраняет его для $i+1$. При $i = n$ весь старый префикс перенесен. После записи x в позицию n и увеличения размера последовательность равна старой последовательности s x в конце, а $0 \leq n+1 \leq C'$.

Контрольный пример на отсчетах

Датчик записывает пять отсчетов: 18,2; 18,4; 18,3; 18,7; 18,6. После первых четырех состояние массива при $C = 4$ имеет $n = 4$. Пятый отсчет требует буфер емкости 8 и перенос четырех значений. Если затем нужны только последние три отсчета для скользящего среднего, повторное удаление нулевого элемента приводит к сдвигам. Для непрерывного потока фиксированного окна удобнее кольцевой буфер емкости 3: новая запись замещает самый старый отсчет, а индекс начала меняется по модулю 3.

Критерии проверки студенческого решения

- Указаны предусловия индекса и инвариант $0 \leq n \leq C$; недоступная часть буфера не выдается клиенту.
- При расширении сохранен порядок элементов и правильно обработана нулевая начальная емкость.
- Разделены худшая стоимость одного APPEND и амортизированная стоимость серии.
- Вывод $\Theta(N)$ для удвоения опирается на сумму переносов, а не только на результаты замера времени.
- Выбор контейнера объяснен операциями рабочей нагрузки, типом элементов и ограничением памяти.